MagicCode Super-Resolution Software Release Notes

Document Version: v2.1.0. Release Date: September 4, 2026.

1. Summary

MagicCode Super-Resolution Software v2.1.0 consolidates the entire Native SDK into a streamlined 3-function public C ABI centered around MC_Enable, MC_Disable, and MC_GetVersion. All legacy separate enable-wrapper libraries and headers have been eliminated; every target platform ships a single unified static library (libmagic_sr.a or libmagic_sr.lib) and a single header (interface/mc_interface.h).

2. Key Changes in v2.1.0

- Public version string is v2.1.0 (MC_GetVersion).
- Streamlined 3-function public ABI: MC_Enable, MC_Disable, MC_GetVersion.
- Unified deliverable: one static library per platform (libmagic_sr.a / libmagic_sr.lib) and one header (interface/mc_interface.h).
- Removed legacy enable wrapper libraries (libmagic_sr_enable.a, libmagic_enable_sr.lib) and mc_enable.h.
- MC_Enable handles implicit handle initialization, dynamic parameter reconfiguration, frame processing, and status reporting in a single entry point.
- MC_Disable safely releases all resources associated with an algorithm handle.
- Removed obsolete session functions: MC_Init, MC_Process, MC_Uninit, MC_Control, control_param_t, and cmd_params_e.
- alg_mode_e: SPATIAL_SPEED_MODE=0, SPATIAL_BALANCED_MODE=1, TEMPORAL_SPEED_MODE=2, TEMPORAL_BALANCED_MODE=3.
- Temporal scaler range is (1, 8]. Spatial remains [1.0, 8.0].
- Both temporal modes default to deriving a missing reactive mask (and approximate transparency on the Metal/Vulkan/GLES Speed path) and to depth+MV history rejection. Explicit masks win per channel.

3. Compatibility

Platform	GPU backends	Library	Temporal
Android	Vulkan, OpenGLES	lib/android/libmagic_sr.a	Temporal GPU yes; CPU temporal no
iOS / iPadOS	Metal	lib/ios/libmagic_sr.a	Temporal GPU yes; CPU temporal no
macOS (Apple Silicon)	Metal	lib/mac_arm/ libmagic_sr.a	Temporal GPU yes; CPU temporal no
macOS (x86_64)	CPU	lib/mac_x86/	Spatial AVX2; Temporal

		libmagic_sr.a	no
Windows	Vulkan, Direct3D 11, OpenGL 4.3	lib/windows/ libmagic_sr.lib	Temporal GPU yes; CPU temporal no

- Operating systems: Windows 10 64-bit and above, Android 8.0 and above, iOS 13.0 and above, iPadOS 13.0 and above, and macOS 12.0 and above.
- Input resolution: 64×64 to 4032×4032.
- CPU SIMD (spatial CPU paths): AVX2 on x86_64, NEON on ARM64. CPU temporal is unavailable.

Temporal Super-Resolution

v2.1.0 provides temporal super-resolution through the unified public C ABI: libmagic_sr.a (Windows: libmagic_sr.lib) + interface/mc_interface.h + MC_Enable / MC_Disable. A single function, MC_Enable, handles implicit handle initialization, dynamic reconfiguration, temporal execution, and execution status query.

Modes

Value	Enumerator	Role
0	SPATIAL_SPEED_MODE	Spatial, throughput first
1	SPATIAL_BALANCED_MODE	Spatial, quality/cost trade-off
2	TEMPORAL_SPEED_MODE	Temporal, throughput first
3	TEMPORAL_BALANCED_MODE	Temporal, canonical FSR-family path

- Temporal scaler_factor is (1, 8]; 1.0 itself is rejected. Spatial remains [1.0, 8.0].
- TEMPORAL_SPEED_MODE on Metal, Vulkan, and OpenGLES currently uses a lightweight Convert → Activate → Upscale pipeline. Direct3D 11 and desktop OpenGL 4.3 Speed use the FSR-family temporal path (same family as Balanced, not a separate customer API).
- TEMPORAL_BALANCED_MODE uses the canonical FSR-family path on every GPU temporal backend. Desktop Balanced (Windows Vulkan / Direct3D 11 / desktop GL; Apple Metal) can run edge-resolve; if enable_sharpening is on, RCAS runs after edge-resolve.
- CPU backends (X86 / NEON) cannot run temporal modes.
- The API accepts (1, 8]. Exact 2× has optimized kernels. Not every scale or device is claimed as device-run.
- Internal pipeline names are implementation detail, not public configuration.

Minimum per-frame inputs

- magic_frame_t.image_in / image_out: caller-owned color and output GPU resources.
- magic_frame_t.frame: non-NULL temporal_frame_t* with struct_size = sizeof(temporal_frame_t).
- temporal_frame_t.depth and motion at input resolution.
- jitter_offset_x/y in input pixels, top-left, +X right, +Y down.
- frame_index / reset_history; camera_near / camera_far / camera_fov_y (radians).

- gpu_context at initial MC_Enable: Vulkan requires physical_device + device; D3D11 requires device; Metal device may be NULL (system default); GL/GLES use the current context (library never makes current).
- Vulkan temporal requires magic_frame_t.command_buffer to be a recording VkCommandBuffer. Metal command_buffer is optional. D3D11 / GL / GLES leave it NULL.
- Create-stage depth_reversed / depth_infinite / hdr_color on input_param_t (all-zero = conventional-Z, finite far, LDR). Immutable for the handle lifetime.

Motion, depth, and masks

- Motion is current → previous. The library never negates MVs.
- motion_vector_scale_x/y is the only MV numeric conversion. (0,0) defaults to (input_width, input_height). Mixed zero is rejected.
- Depth is GPU device depth in [0, 1]. Linear view-space depth is not accepted.
- reactive / transparency are optional. Explicit caller textures win per channel.
- When omitted, the library derives an approximate mask. That is not a semantic transparency mask.
- Speed on Metal/Vulkan/GLES may derive both missing reactive and an approximate transparency. FSR-family paths (Balanced on all backends; Speed on D3D11/desktop GL) derive missing reactive only and bind an internal zero transparency texture.
- Both temporal modes enable depth+MV history rejection by default.
- Diagnostic only: MAGICSR_TAAU_AUTO_MASK=0 and MAGICSR_TAAU_HIST_REJECT=0 can turn those defaults off. They are not a customer configuration path.

4. Migration Notes

- Replace legacy headers with v2.1.0 interface/mc_interface.h.
- Link only libmagic_sr.a (or libmagic_sr.lib); drop any references to libmagic_sr_enable.a or libmagic_enable_sr.lib.
- Update API calls: replace MC_Init and MC_Process with unified MC_Enable(&handle, &frame, &param, &status). Replace MC_Uninit with MC_Disable(handle).
- Remove control_param_t; pass updated input_param_t to MC_Enable to reconfigure parameters dynamically.
- Review Privacy Policy and Data Reporting Notice before distributing applications that include the SDK. Those legal documents are unchanged by this technical release.

5. Known Limitations

- Not every backend, scale, or device class is device-run. Distinguish API support from verified configurations.
- Derived masks are approximations. Omitting transparency is not equivalent to a semantic transparency mask.
- Vulkan temporal requires a recording command buffer and explicit layouts; the library does not begin, end, or submit it.

- Performance depends on device, driver, backend, resolution, scale, and motion quality.

6. Validation Snapshot

API contracts are defined by interface/mc_interface.h and the GPU host implementations. Verified with automated Release builds on Android arm64-v8a, iOS arm64, macOS Apple Silicon arm64, and macOS x86_64.